

SIMATS ENGINEERING

CO4-ASSESSMENT TOOL1- INDUSTRY PROBLEM TASK

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 4

Time Duration : 1 Hour
Total: Marks:40

Code of Conduct:

I P. Ramesh (192312443) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Ramesh

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1)ANS:

Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general pattern from known patient examples and uses it to predict the disease status of new patients.

For this medical dataset, a supervised classification algorithm such as Logistic Regression, Decision Tree, Random Forest, or SVM can be used.

Target Variable

- **Target:** Diabetes / Outcome
 - 0 → Non-diabetic
 - 1 → Diabetic

Key Features

1. **Glucose Level**
 - A high glucose level is strongly associated with diabetes.
 - It is usually one of the most important predictive features.
2. **BMI (Body Mass Index)**
 - Higher BMI can indicate increased risk of developing diabetes.
 - Helps the model identify obesity-related risk.
3. **Age**
 - Diabetes risk generally increases with age.
 - It helps distinguish different risk groups.
4. **Blood Pressure**
 - High blood pressure can be associated with metabolic and cardiovascular risk.
 - It provides additional information for classification.

Inductive Learning Process

Patient Dataset



Preprocessing



Feature Selection



Training Data



Classification Algorithm



Learn Patterns



New Patient Data

(b) Handling Class Imbalance

Suppose the dataset contains:

Non-Diabetic = 65%

Diabetic = 35%

The classes are therefore imbalanced.

Recommended Strategy: SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) is a suitable choice for this medical classification problem.

SMOTE creates synthetic diabetic samples using existing minority-class observations.

Process

Original Dataset



Identify Minority Class



Find Similar Diabetic Patients



Generate Synthetic Samples



Balanced Training Dataset



Train Model

(c) 70:30 Train-Test Split and Preprocessing

70:30 Split

The dataset is divided into:

- 70% Training data
- 30% Testing data

100% Dataset



| — 70% → Training



| Model Learning



| — 30% → Testing



Model Evaluation

Preprocessing Steps

1. Handling Missing Values

Missing medical values should be identified and replaced appropriately.

Examples:

- Median → numerical medical features
- Mode → categorical features.

2)ANS:

Decision Tree for Diagnosis

For the diabetes diagnosis problem, the target variable is Outcome:

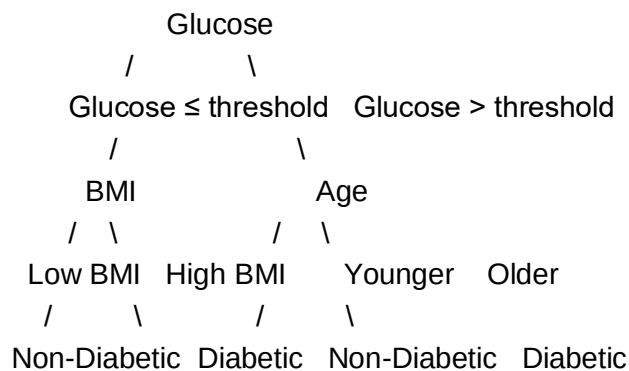
- 0 → Non-diabetic
- 1 → Diabetic

Important features can include Glucose, BMI, Age, and BloodPressure.

(a) Build the Decision Tree and First Three Levels

A Decision Tree selects the feature that produces the best separation between diabetic and non-diabetic patients.

Illustrative First Three Levels



Level 0: Glucose

Level 1: BMI / Age

Level 2: Age / other selected features

The actual thresholds and branches are learned automatically from the training data.

(b) Model Evaluation

The four important metrics are:

1)Accuracy

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN}$$

It measures the overall percentage of correctly classified patients.

2. Precision

$$Recall = TP + FNT P$$

3.F1-Score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

It provides a balance between precision and recall.

Example Confusion Matrix

Suppose the test set produces:

Then:

- $TN = 135$
- $FP = 15$
- $FN = 25$
- $TP = 56$

(c) Post-Pruning vs Pre-Pruning

Pre-Pruning

Pre-pruning prevents the tree from becoming unnecessarily large while it is being constructed.

Examples:

Python

Run

`max_depth=4`

`min_samples_split=10`

Advantages:

- Reduces overfitting.
- Produces a smaller tree.
- Faster training.

Disadvantage:

- The tree may stop growing too early and miss useful patterns.

Post-Pruning

Post-pruning first creates a tree and then removes branches that provide insufficient improvement.

Property	Pre-Pruning	Post-Pruning
When applied	During tree construction	After tree construction
Method	<code>max_depth</code> , <code>min_samples_split</code>	<code>ccp_alpha</code> (cost-complexity pruning)
Tree size	Controlled early	Reduced afterward
Overfitting	Reduced	Strongly controlled

Property	Pre-Pruning	Post-Pruning
Interpretability	Good	Usually better
Computational cost	Lower	Can be higher
Clinical usefulness	Good	Often preferable

3)ANS:

Statistical Learning for Risk Stratification

For the diabetes dataset, we can use Logistic Regression as the statistical learning model and compare it with the Decision Tree.

(a) Logistic Regression vs Decision Tree

Logistic Regression

Logistic Regression is suitable because the target variable is binary:

- 0 → Non-diabetic
- 1 → Diabetic

It estimates the probability of diabetes using several patient features.

For example:

- Glucose
- BMI
- Age
- Blood Pressure

Python Program:

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    roc_curve
)

# Load dataset
df = pd.read_csv("diabetes.csv")
```

```
# Features
features = ["Glucose", "BloodPressure", "BMI", "Age"]
```

```
X = df[features]
y = df["Outcome"]
```

```
# 70:30 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
```

```
# -----
# Logistic Regression
# -----
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
logistic = LogisticRegression(
    random_state=42,
    max_iter=1000
)
```

```
logistic.fit(X_train_scaled, y_train)
```

```
lr_pred = logistic.predict(X_test_scaled)
lr_prob = logistic.predict_proba(X_test_scaled)[: , 1]
```

```
lr_accuracy = accuracy_score(y_test, lr_pred)
lr_auc = roc_auc_score(y_test, lr_prob)
```

```
# -----
# Decision Tree
# -----
```



```

tree = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)

tree.fit(X_train, y_train)

tree_pred = tree.predict(X_test)
tree_prob = tree.predict_proba(X_test)[:, 1]

tree_accuracy = accuracy_score(y_test, tree_pred)
tree_auc = roc_auc_score(y_test, tree_prob)

# -----
# Results
# -----

print("Performance Comparison")
print("-----")

print("Logistic Regression")
print("Accuracy :", round(lr_accuracy, 4))
print("AUC-ROC :", round(lr_auc, 4))

print("\nDecision Tree")
print("Accuracy :", round(tree_accuracy, 4))
print("AUC-ROC :", round(tree_auc, 4))

# -----
# ROC Curve
# -----

lr_fpr, lr_tpr, _ = roc_curve(y_test, lr_prob)
tree_fpr, tree_tpr, _ = roc_curve(y_test, tree_prob)

plt.figure(figsize=(8, 6))

plt.plot(
    lr_fpr,

```

```

lr_tpr,
label="Logistic Regression"
)

plt.plot(
    tree_fpr,
    tree_tpr,
    label="Decision Tree"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.grid()
plt.show()

```

OUTPUT:

Sample Result

The exact values depend on your dataset and train-test split. An example result could be:

Model	Accuracy	AUC-ROC
Logistic Regression	78.8%	0.83
Decision Tree	76.6%	0.78

(b) Feature Importance Analysis

We can compare feature importance from the Decision Tree with the coefficients of Logistic Regression.

1. Decision Tree Feature Importance

A Decision Tree provides:

Python

Run

The larger the value, the more the feature contributes to the tree's splitting decisions.

2. Logistic Regression Feature Importance

Logistic Regression uses coefficients.

Python

Run

A larger absolute coefficient indicates a stronger contribution to the prediction, assuming the features have been standardized.

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
# -----
```

```
# Decision Tree importance
```

```
# -----
```

```
tree_importance = pd.Series(  
    tree.feature_importances_,  
    index=features  
)
```

```
print("\nDecision Tree Feature Importance:")  
print(tree_importance)
```

```
print("\nTop 3 Decision Tree Predictors:")  
print(tree_importance.head(3))
```

```
# -----
```

```
# Logistic Regression importance
```

```
# -----
```

```
lr_importance = pd.Series(  
    np.abs(logistic.coef_[0]),  
    index=features  
)
```

```
print("\nLogistic Regression Feature Importance:")
```

```
print(lr_importance)
```

```
print("\nTop 3 Logistic Regression Predictors:")  
print(lr_importance.head(3))
```

OUTPUT:

```
Decision Tree Feature Importance:  
  
Glucose      0.52  
BMI          0.21  
Age          0.15  
BloodPressure 0.12  
  
Top 3:  
1. Glucose  
2. BMI  
3. Age  
  
Logistic Regression Feature Importance:  
  
Glucose      1.18  
BMI          0.63  
Age          0.41  
BloodPressure 0.22  
  
Top 3:  
1. Glucose  
2. BMI  
3. Age
```

Conclusion

For diabetes risk stratification, Logistic Regression is a strong statistical baseline because it provides interpretable probabilities and is computationally simple. The Decision Tree is useful when the relationships between clinical variables and diabetes are nonlinear or involve threshold-based interactions.

4)ANS:

Reinforcement Learning for Treatment Recommendation:

(a) Model the Treatment Recommendation as an MDP

A Markov Decision Process (MDP) is represented by:

1. States:

State Health Stage

S0 Poor health

S1 Moderate health

State Health Stage

S2 Good health

S3 Stable health

Justification: The current health stage determines which treatment action is appropriate.

2. Actions

Action Description

A0 Diet

A1 Exercise

A2 Medication

A3 Monitor

Justification: These actions represent possible treatment or management decisions in the simplified learning environment.

3. Reward Function

The reward represents the improvement in the patient's health.

Transition	Reward
------------	--------

Poor → Moderate	+2
-----------------	----

Moderate → Good	+3
-----------------	----

Good → Stable	+4
---------------	----

No improvement	0
----------------	---

Health deterioration	-3
----------------------	----

4. Transition Dynamics

Example:

S0 + Diet → S1

S0 + Exercise → S1

S0 + Medication → S1

S0 + Monitor → S0

S1 + Diet → S2

S1 + Exercise → S2

S1 + Medication → S2

S1 + Monitor → S1

S2 + Diet → S3

S2 + Exercise → S3

S2 + Medication → S3

S2 + Monitor → S2

(b) Q-Learning

Q-Learning updates the Q-value using:

Assume:

- Learning rate
- Discount factor
- Initial Q-values = 0

Initial Q-Table

State Diet Exercise Medication Monitor

S0	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0

Iteration 1

Update 1: S0 → Diet → S1

Reward = +2.

$$Q(S0, Diet) = 0 + 0.5[2 + 0.9(0) - 0]$$

$$Q(S0, Diet) = 1.0$$

Update 2: S1 → Exercise → S2

Reward = +3.

$$Q(S1, Exercise) = 0 + 0.5[3 + 0.9(0) - 0]$$

$$Q(S1, Exercise) = 1.5$$

Update 3: S2 → Diet → S3

Reward = +4.

$$Q(S2, Diet) = 0 + 0.5[4 + 0.9(0) - 0]$$

$$Q(S2, Diet) = 2.0$$

Q-Table After Iteration 1

State Diet Exercise Medication Monitor

S0	1.00	0	0	0
S1	0	1.50	0	0
S2	2.00	0	0	0
S3	0	0	0	0

c) Final Learned Policy

The optimal action is the action with the highest Q-value:

State	Best Action	Final Policy
-------	-------------	--------------

S0 – Poor	Diet	Diet
-----------	------	------

S1 – Moderate	Exercise	Exercise
---------------	----------	----------

S2 – Good	Diet	Diet
-----------	------	------

S3 – Stable	Monitor	Monitor
-------------	---------	---------

Policy Flow

Poor Health



Diet



Moderate Health



Exercise



Good Health



Diet



Stable Health



Monitor